

Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}

Programming

File Management



Prof. Nilesh Gambhava

Computer Engineering Department,
Darshan Institute of Engineering & Technology, Rajkot





File management is what you have, and how you want to manipulate it. - Anonymous



Why File Management?

- ▶ In real life, we want to store data permanently so that later we can retrieve it and reuse it.
- ▶ A file is a collection of characters stored on a secondary storage device like hard disk, or pen drive.
- ▶ There are two kinds of files that programmer deals with:
 - **Text Files** are human readable and it is a stream of plain English characters
 - **Binary Files** are computer readable, and it is a stream of processed characters and ASCII symbols

Text File

Hello, this is a text file. Whatever written here can be read easily without the help of a computer.

Binary File

```
11010011010100010110111010
10111010111010011010100010
11011101010111010111010011
```

File Opening Modes

- ▶ We can perform different operations on a file based on the file opening modes

Mode	Description
r	Open the file for reading only. If it exists, then the file is opened with the current contents; otherwise an error occurs.
w	Open the file for writing only. A file with specified name is created if the file does not exist. The contents are deleted, if the file already exists.
a	Open the file for appending (or adding data at the end of file) data to it. The file is opened with the current contents safe. A file with the specified name is created if the file does not exist.
r+	The existing file is opened to the beginning for both reading and writing.
w+	Same as w except both for reading and writing.
a+	Same as a except both for reading and writing.

Note: The main difference is w+ truncate the file to zero length if it exists or create a new file if it doesn't. While r+ neither deletes the content nor create a new file if it doesn't exist.

File Handling Functions

- ▶ Basic file operation performed on a file are opening, reading, writing, and closing a file.

Syntax	Description
<code>fp=fopen(file_name, mode);</code>	This statement opens the file and assigns an identifier to the FILE type pointer fp. Example: <code>fp = fopen("printfile.c", "r");</code>
<code>fclose(filepointer);</code>	Closes a file and release the pointer. Example: <code>fclose(fp);</code>
<code>fprintf(fp, "control string", list);</code>	Here fp is a file pointer associated with a file. The control string contains items to be printed. The list may includes variables, constants and strings. Example: <code>fprintf(fp, "%s %d %c", name, age, gender);</code>

File Handling Functions

Syntax	Description
<code>fscanf(fp, "control string", list);</code>	Here <code>fp</code> is a file pointer associated with a file. The control string contains items to be printed. The list may includes variables, constants and strings. Example: <code>fscanf(fp, "%s %d", &item, &qty);</code>
<code>int getc(FILE *fp);</code>	<code>getc()</code> returns the next character from a file referred by <code>fp</code>; it require the FILE pointer to tell from which file. It returns EOF for end of file or error. Example: <code>c = getc(fp);</code>
<code>int putc(int c, FILE *fp);</code>	<code>putc()</code> writes or appends the character <code>c</code> to the FILE <code>fp</code>. If a <code>putc</code> function is successful, it returns the character written, EOF if an error occurs. Example: <code>putc(c, fp);</code>

File Handling Functions

Syntax	Description
<code>int getw(FILE *pvar);</code>	<code>getw()</code> reads an integer value from <code>FILE</code> pointer <code>fp</code> and returns an <code>integer</code> . Example: <code>i = getw(fp);</code>
<code>putw(int, FILE *fp);</code>	<code>putw</code> writes an integer value read from terminal and are written to the <code>FILE</code> using <code>fp</code> . Example: <code>putw(i, fp);</code>
<code>EOF</code>	<code>EOF</code> stands for "End of File". <code>EOF</code> is an integer defined in <code><stdio.h></code> Example: <code>while(ch != EOF)</code>

Write a C program to display content of a given file.

Program

```
1  #include <stdio.h>
2  void main()
3  {
4      FILE *fp; //p is a FILE type pointer
5      char ch; //ch is used to store single character
6      fp = fopen("file1.c","r"); //open file in read mode and store file pointer in p
7      do { //repeat step 9 and 10 until EOF is reached
8          ch = getc(fp); //get character pointed by p into ch
9          putchar(ch); //print ch value on monitor
10     }while(ch != EOF); //condition to check EOF is reached or not
11     fclose(fp); //free up the file pointer pointed by fp
12 }
13
```


Write a C program to copy a given file.

Program

```
1 #include <stdio.h>
2 void main()
3 {
4     FILE *fp1, *fp2; //p and q is a FILE type pointer
5     char ch; //ch is used to store temporary data
6     fp1 = fopen("file1.c","r"); //open file "file1.c" in read mode
7     fp2 = fopen("file2.c","w"); //open file "file2.c" in write mode
8     do { //repeat step 9 and 10 until EOF is reached
9         ch = getc(fp1); //get character pointed by p into ch
10        putc(ch, fp2); //print ch value into file, pointed by pointer q
11    }while(ch != EOF); //condition to check EOF is reached or not
12    fclose(fp1); //free up the file pointer p
13    fclose(fp2); //free up the file pointer q
14    printf("File copied successfully...");
15 }
```

File Positioning Functions

- ▶ `fseek`, `ftell`, and `rewind` functions will set the file pointer to new location.
- ▶ A subsequent read or write will access data from the new position.

Syntax	Description
<code>fseek(FILE *fp, long offset, int position);</code>	<code>fseek()</code> function is used to move the file position to a desired location within the file. <code>fp</code> is a <code>FILE</code> pointer, <code>offset</code> is a value of datatype <code>long</code>, and <code>position</code> is an <code>integer</code> number. Example: <code>/* Go to the end of the file, past the last character of the file */</code> <code>fseek(fp, 0L, 2);</code>
<code>long ftell(FILE *fp);</code>	<code>ftell</code> takes a file pointer and returns a number of datatype <code>long</code>, that corresponds to the current position. This function is useful in saving the current position of a file. Example: <code>/* n would give the relative offset of the current position. */</code> <code>n = ftell(fp);</code>

File Positioning Functions

Syntax	Description
<code>rewind(fp);</code>	<code>rewind()</code> takes a file pointer and resets the position to the start of the file. Example: <code>/* The statement would assign 0 to n because the file position has been set to the start of the file by rewind. */</code> <pre>rewind(fp);</pre>

Write a C program to count lines, words, tabs, and characters

Program

```
1 #include <stdio.h>
2 void main()
3 {
4     FILE *p;
5     char ch;
6     int ln=0,t=0,w=0,c=0;
7     p = fopen("text1.txt","r");
8     ch = getc(p);
9     while (ch != EOF) {
10         if (ch == '\n')
11             ln++;
12         else if(ch == '\t')
13             t++;
14         else if(ch == ' ')
15             w++;
16         else
```

Program (contd.)

```
17             c++;
18
19         ch = getc(p);
20     }
21     fclose(p);
22     printf("Lines = %d, tabs = %d, w
ords = %d, characters = %d\n",ln, t,
w, c);
23 }
```

Output

Lines = 22, tabs = 0, words = 152, characters = 283

Practice Programs

- 1) Write a C program to write a string in file.
- 2) A file named data contains series of integer numbers. Write a C program to read all numbers from file and then write all the odd numbers into file named "odd" and write all even numbers into file named "even". Display all the contents of these file on screen.
- 3) Write a C program to read name and marks of n number of students and store them in a file.
- 4) Write a C program to print contents in reverse order of a file.
- 5) Write a C program to compare contents of two files.
- 6) Write a C program to copy number of bytes from a specific offset to another file.
- 7) Write a C program to convert all characters in UPPER CASE of a File.



Thank you

